

Runbook — Invoice & Receipt → Structured Data

The artefact that makes a maintenance retainer worth buying. Handed over with every build.

Written for whoever operates this day to day, not for a developer. If you can find the workflow in n8n and open a node, you can follow this.

What it does

Reads supplier documents — invoices, till receipts, subscription bills — and turns each one into a single row: supplier, reference, date, currency, net, VAT, gross. Before anything is exported it re-checks the numbers in plain code. Documents that pass go to **Ready to Export**. Documents it is not confident about go to **Flag for Review**, each carrying the specific reason.

It is designed to tell you when not to trust it. That is the whole point of it. A run that flags three documents out of forty has done its job; a run that flags nothing and quietly books a wrong VAT figure has not.

Normal operation

One row out per document in. A steady trickle of flagged documents is **normal and healthy** — till receipts genuinely do not carry invoice numbers, and the workflow says so rather than inventing one. Expect these routinely:

- No document reference (common on till receipts)
- Net not stated on document; derived as gross minus VAT

Neither is a fault. Both are the system reporting what it did.

What is worth noticing is a *change* in the flag rate. If a supplier you have processed for months starts flagging, their document layout has changed — that is useful information, and it is the sort of thing the monthly check is for.

The five checks it runs

Every document is checked in code, not by the model:

#	CHECK	FLAGS WHEN
1	Arithmetic	net + VAT does not equal gross, allowing 2p of rounding drift
2	Missing net	Net is absent but gross is present — it derives net as gross – VAT and records that it did
3	VAT rate plausibility	The implied rate is not within 1.5 points of a UK band (0%, 5%, 20%)
4	Date	The date cannot be parsed into an ISO date an accounts system will accept
5	Required fields	Supplier name, gross total, or document reference is missing

What breaks it, and what to do

SYMPTOM	LIKELY CAUSE	FIX
Everything is suddenly flagged, or the run stops at <i>Extract Fields</i>	The Google AI Studio key has hit its free-tier limit or been revoked. The node retries three times, five seconds apart, then gives up	Check the key at https://aistudio.google.com/apikey . A 429 in the execution log means rate limit, not a broken workflow — wait and re-run
Dates land one day early or late	A date format the parser read the other way round	Numeric dates are read UK day-first : 07/08/2026 is 7 August, not 8 July. If a supplier issues US-format dates, that supplier needs a rule of its own — tell me and I will add one
A new supplier's documents come back mostly empty	Their layout is unlike anything the extractor has seen, and the prompt forbids guessing	This is the safe failure, not a broken one. Send me two examples and the layout gets added
Rows appear with no VAT and an "Unusual VAT rate" flag	A non-UK or mixed-rate document, or a zero-rated supplier	Check the document. If the supplier is legitimately outside the UK VAT bands, the band list is one line in <i>Validate & Normalise</i> and can be widened
Nothing runs at all, and there are no errors	The workflow was deactivated by an edit	Re-activate it. Editing a workflow can silently deactivate it — this catches everyone once
Two rows for one document	The source was triggered twice	Check the source node, not the extractor. In a deployment this is where a duplicate check belongs, and it is worth adding before it becomes a habit
Currency is wrong	It was inferred from a symbol rather than stated	€ and \$ are ambiguous across countries. If a supplier always bills in one currency, that can be pinned per-supplier

Re-running a document

Safe to do. Nothing is written until the export branch runs, and re-reading a document produces the same result — the model runs at temperature 0, so the same input gives the same output. If you have already exported a row and re-run it, you will get a second row; delete the first.

What to do with a flagged document

Open **Flag for Review** and read `review_reasons`. It names the check that failed, so you are looking for one specific thing rather than re-reading the whole document. Correct the row by hand and export it. If the same reason appears repeatedly for the same supplier, that is worth telling me about — it is usually a five-minute fix that removes the flag permanently.

What it will not do

Stated plainly so there are no surprises:

- **It does not send or file anything on its own.** In this repository it ends at two Set nodes. In a deployment the export branch is wired to your sheet or accounting system, and that is a decision you make at setup — including whether derived values are written automatically or held for approval.

- **It does not read handwriting**, and it is unreliable on badly-lit photographs of crumpled receipts. Flatbed scans and PDFs are what it is good at.
- **It does not decide what is tax-deductible**, allocate nominal codes, or make any accounting judgement. It reads what the document says.

What monthly maintenance covers

Watching the flag rate for changes, adding supplier layouts as new suppliers appear, rotating the API key when needed, keeping up with model version changes, testing n8n upgrades against this workflow before you take them, and adjusting where flagged documents are sent.

The test suite (`node test/logic-test.mjs`) is run after every change, so a fix to one supplier's date format cannot quietly break another's arithmetic.

Hosting

Delivered as version-controlled JSON that runs on **your** n8n instance, under your credentials and your Google AI Studio key. You own it and you host it — no lock-in, and it keeps running whatever happens to me.

This runbook describes the demonstration workflow in this repository. A client build gets the same document with the supplier list, destinations and contact routes filled in.

github.com/benfoster1997/n8n-portfolio · Self-initiated demonstration project, not client work.